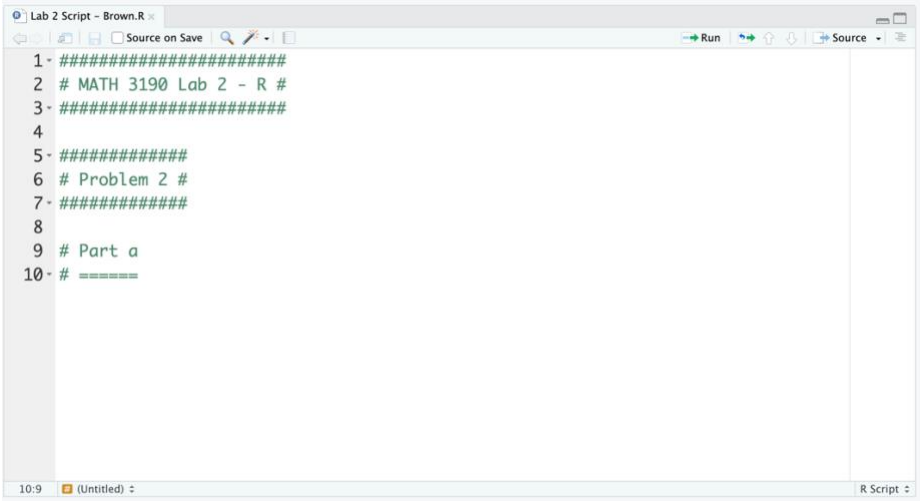
Lab 2 – R

This assignment should be added to your GitHib page in the Math_3190_Assignments repo in the Labs folder in the Lab_2 folder. In addition, the R script file should be submitted to Canvas in the Lab 2 assignment.

1. **Setup**.
    a. Install **R** and RStudio or Positron on your computer if you have not done so already.

    b. Once you have **R** and your IDE installed, open your IDE and create a new R script. Save this file in the Math_3190_Assignments directory you created in Lab 1 in the Labs/Lab_2 folder. Name this script “Lab 2 - Last” where Last is your last name. So, for me, the file will be called “Lab 2 - Brown”. The extension for this file should be .R. So, the file name should be **Lab 2 - Last.R**.

    c. In your R script file, do all of the following. Before each part, put a comment in your script indicating that you are doing that part. As an example, see this image, but your commenting style certainly does not need to match mine:



Note: it is considered bad programming practice to have lines that are too long. That includes comments. There is a vertical line on the right side in every **R** script (seen in the picture above below the word "Source". It is good practice to not surpass that vertical line. If your line of code is too long, then you can press "Enter" or "Return" on your keyboard to wrap it to the next line.

2. **Data structures**
    a. Install the **tidyverse** (if you have not already) and load it using the **library()** function.

    b. Create a character vector called **employee_names** with "Alice", "Bob", "Charlie", "Diana", "Edward" in it.

c. Create a numeric vector called **monthly_hours** with the values 160, 145, 180, 120, 155.

d. Combine these vectors into a data frame called **work_data**.

e. Then combine the vectors into a tibble called **work_tibble**. Print both the **work_data** and the **work_tibble** to see some of the differences between the two data objects.

f. Using either **work_tibble or work_data**, print only the third and fifth row of the dataset using one command.

g. Now isolate the first column using two different methods.

h. Take the average of the **monthly_hours** column from either **work_tibble** or **work_data** (not just using the vector you defined earlier).

3. **Functions**.
   a. The formula for converting Fahrenheit to Kelvin is $K = \frac{5}{9}(F + 459.67)$. Write a function called **to_kelvin** that takes a numeric input **Fahrenheit** and test it using the Fahrenheit values 0 and 100.

   b. Create a function called **fuel_cost** that takes **miles**, **mpg**, and **price_per_gallon** as inputs and returns the total cost of the trip. Then use it to calculate the cost for a 400-mile trip in a car that gets 32 MPG when gas is $3.25/gallon.

4. **Data wrangling**. The **mpg** dataset contains fuel economy data from 1999 and 2008 for 38 popular models of cars. Type **?mpg** after loading the **tidyverse** to learn more about this data set. In a single expression using pipes, perform the following on the **mpg** dataset:
   - Filter for cars made only by Audi, Ford, or Toyota. The **%in%** operator is useful here.
   - Mutate a new column called **combined_mpg** which is the average of city and highway mileage.
   - Select only the columns **manufacturer**, **model**, **cyl**, **class**, and **combined_mpg**.
   - Group by the **class** of the car.
   - Summarize the data to find the mean and standard deviation of **combined_mpg** mileage for each **class**.
   - Arrange it by the mean **combined_mpg** (from best to worst).

Once all of that is done, leave a comment in the code indicating which class had the best average combined MPG.

5. **Graphing with ggplot2**. We will use the **midwest** dataset, which contains demographic information on **midwest** counties and is included with the **ggplot2** library.

a. Create a scatter plot with **poptotal** (total population) on the x-axis and **area** on the y-axis. Add a trend line using **geom_smooth(method = "lm", se = F, formula = "y ~ x")**.

b. Copy your code from part a. Now, because the populations differ by orders of magnitude, add **scale_x_log10()** to your plot and color the points by state. Also give the graph appropriate labels on the x and y axes and give it a title.

c. Create a side-by-side boxplots of **percollege** (percentage of college educated) across the different **state** categories. Fill the boxes with a color (or colors) of your choice and set the **alpha** to 0.6. Add **geom_jitter()** on top of the boxplots so you can see the individual county data points. Set the jitter **color** to one of your choice, the **size** to 0.5, and the **width** to 0.1.

d. Create a density plot of **percbelowpoverty**. Use facet grid for **state** to see the distribution for each state separately. Change the theme to black and white and add the title "Poverty Rates Across Midwest States". Finally, modify the theme to make the title size 18 and bold.

e. Now, use the **tidyverse** to take the **midwest** data, group it by the **state** variable, find the number of counties in each state with the **summarize()** function, and then using that summary, create a bar chart showing the count of counties in each state. Change the fill color of the bars to "steelblue" and the color to "black". Also make the axis labels look nice.

6. **Data import**.
    a. On the class GitHub page is a Data folder. In that folder is a dataset called **treeseeds.txt**. Download that file and read it into **R**. Make sure to name it when you read it in. Then use the **head()** function to print the first six rows of the dataset.

    b. Also on the GitHub page is the **blood_pressure.txt** data file. Read that into **R** and print the first six rows.

    c. Once more, there is a data file called **Concrete_Data.xls** on GitHub. Read that into **R** and print the first six rows.

7. **for loop**. **R** uses vectorization, which means that many functions work on every element of a vector at once. If we can apply an expression to the entire vector, it usually speeds up the code and often considerably. However, it is also important to know how to use for loops to handle one element of a vector at a time. If you are unfamiliar with loops and conditionals in **R**, there is some explanation about them on this website. Take some time to read through that.

    a. Write a for loop that iterates through the **monthly_hours** vector you created earlier. Inside the loop, use an if/else statement. If hours are greater than 150, print the employee's name and the phrase "Overtime Eligible". Otherwise, print the name and

"Standard Hours". The **print()** and **paste()** functions are useful here.

b. Now let's do the same thing without the loop using the **tidyverse**. Use **mutate()** and the **ifelse()** function or the **case_when()** function on your **work_data** or **work_tibble** dataset to create a new column called **status** that contains the same labels ("Overtime Eligible" or "Standard Hours") depending on if the work hours are greater than 150 or not.